JQ
JSON E CSV

*O `curl … | grep '"ip"' | cut -d'"' -f4` funcionou por duas semanas. Aí a API passou a mandar a resposta **minificada**, tudo numa linha — e o `grep`, que decide por linha, viu o documento inteiro como **uma** linha. Consertado o corte, quebrou de novo num campo com **aspa escapada**; e de novo quando a API **trocou a ordem das chaves**, coisa que o JSON permite porque objeto é **mapa, não sequência**. Nenhum é bug do script: os três vêm da mesma decisão errada.*

069
GUIAS LINUX

O MODELO: UM FILTRO QUE TRANSFORMA VALORES EM VALORES



Tudo no `jq` é esse encadeamento. O `|` interno funciona como o do shell, só que carregando **valores JSON** em vez de bytes: `.[] | select(.porta==22) | .ip` lê-se "para cada item da lista, se a porta for 22, devolva o campo `ip`". A diferença crucial é que um filtro pode emitir **zero, um ou vários** valores para cada entrada — o `.[]` emite vários, o `select` emite zero ou um, e é dessa aritmética que sai todo o poder da ferramenta. **E o que sai do `jq` ainda é JSON**: cadeias vêm entre aspas e escapadas. Para alimentar o próximo comando do pipeline é quase sempre `jq -r` (*raw*) — **a opção mais esquecida** e a causa número um de nome de arquivo com aspas no meio.

CSV É UMA ARMADILHA — E OS SELETORES QUE RESOLVEM O RESTO

linha bem-comportada: `10.0.0.1,web,producao`

`cut -d, -f2`	`web`	correto

linha real: `10.0.0.1,"servidor web, filial SP",producao`

`cut -d, -f2`	`"servidor web`	cortou ao meio
`awk -F, '{print $2}'`	`"servidor web`	mesmo erro
`mlr --icsv --ojson`	`"servidor web, filial SP"`	correto

A especificação do CSV permite **vírgula dentro de campo entre aspas**, **aspas duplicadas** para representar uma aspa literal (`""`) e até **quebra de linha dentro de um campo** — o que arruína a premissa "um registro por linha" de que dependem `grep`, `cut`, `sed` e `awk` **ao mesmo tempo**. Exportação de planilha, de banco e de ferramenta comercial traz as três coisas rotineiramente. As saídas: `mlr` (Miller), `csvkit` ou uma linguagem com biblioteca de CSV; ou converta uma vez para JSON/TSV e trabalhe dali. **Para *gerar* CSV, `jq -r '… | @csv'` produz a citação correta de graça.** Só trate CSV com `cut` quando você controlar quem o gerou **e** souber que não há vírgula, aspa nem quebra de linha nos valores.

Os outros formatos têm suas ferramentas. `yq -o=json '.' config.yaml` converte YAML e devolve o problema ao `jq` — o `yq` moderno é, em essência, um `jq` que fala YAML, com a mesma sintaxe de filtros. **Cuidado:** existem **dois projetos com o nome `yq`** (o de Mike Farah, em Go, e o `yq` em Python), com linhas de comando **incompatíveis** — saiba qual está instalado. Para XML, **`xmlstarlet`** usa XPath e é a ferramenta certa para o `-oX` do `nmap`.

NAVEGAR

`.host.rede.ip`	campo aninhado · `."chave com espaço"` quando não é identificador
`.[] · .[0] · .[-1]`	cada elemento · o primeiro · **o último** · `.[2:5]` fatia
`.talvez? · ..`	não falha se o tipo não permitir · **descida recursiva**: todos os valores, em qualquer nível
`.inexistente`	devolve `null` **em silêncio**, não erro — um erro de digitação vira uma coluna de `null`. Exija com `has("chave")` ou `.chave // error("faltou")`
`a // b`	valor padrão quando o lado esquerdo é `null` ou `false`

FILTRAR, TRANSFORMAR, AGREGAR

`select(cond)`	emite **zero ou um** · `select(.ip | startswith("10."))` · `select(.tags | index("prod"))`
`map(f) vs .[]|f`	`map` **devolve um array;** `.[] |` **devolve vários valores soltos.** Escolher errado é a origem de `Cannot iterate over null` e dos colchetes a mais
`group_by · add`	`group_by(.porta) | map({porta: .[0].porta, n: length})` · `sort_by`, `unique_by`, `any`, `keys`, `to_entries`
`{a: .x} · \(…)`	constrói objeto novo · interpola em cadeia: `"\(.ip):\(.porta)"`

SAIR DO JSON E VOLTAR AO PIPELINE

`-r · -c`	sem aspas · compacto, **um documento por linha** (NDJSON)
`@csv @tsv @uri`	citação e escape **corretos** de graça · `@base64`
`-s (--slurp)`	junta o fluxo **num array** — sem ele, cada linha do NDJSON é processada isoladamente e `length` responde por documento
`-R · -n`	lê texto comum (converte log em JSON) · não lê entrada, só produz
`-e (--exit-status)`	sai com **1** quando o resultado é `null` / `false`: é o `grep -q` do JSON, para usar em `if`
`--stream`	para arquivo maior que a memória — desconfortável e raro; o usual é converter para NDJSON uma vez com `jq -c '.[]'`

Valor do shell entra com `--arg`, nunca por interpolação — a mesma regra do `-v` do `awk`. `jq --arg alvo "$IP" '.[] | select(.ip == $alvo)'` é seguro (e `--argjson lim 1024` passa número, booleano ou objeto). Já `jq "select(.host == \"$HOST\")"` **quebra na primeira aspa e é injeção de código na segunda** — e o valor costuma vir do próprio alvo: nome de host, cabeçalho, campo de resposta. **Dado que veio do alvo é dado hostil por definição.** Pelo mesmo motivo, **não redija relatório com `sed` sobre JSON**: é possível e falha em silêncio no primeiro campo aninhado — e redação que falha é vazamento. O certo percorre a estrutura inteira: `jq '(.. | .password? , .token? , .cookie?) |= "REDIGIDO"' bruto.json > entrega.json`.

RESUMO RÁPIDO

- ✓ **JSON não é texto orientado a linhas.** Espaço em branco é livre, a estrutura é aninhada e os valores têm escapes — daí `grep`, `cut` e `sed` funcionarem até deixarem de funcionar, sem aviso.
- ✓ **O `jq` é uma linguagem de filtro:** recebe um fluxo de valores, aplica a expressão e emite outro fluxo. O `|` interno compõe filtros como o do shell compõe comandos.
- ✓ **`.[]` explode uma coleção em vários valores; `map(f)` devolve um array.** Escolher o errado é a origem das saídas com colchetes a mais ou a menos.
- ✓ **`-r` é quase obrigatório quando a saída alimenta outro comando** — sem ele, as cadeias saem com aspas de JSON.
- ✓ **Campo ausente devolve `null` em silêncio.** Use `has()`, `//` para padrão ou `// error(…)` para exigir presença.
- ✓ **Valores do shell entram com `--arg` e `--argjson`**, nunca por interpolação.
- ✓ **`@csv` e `@tsv` geram saída citada corretamente**, e `-e` faz o `jq` servir de condição em `if`.
- ✓ **CSV com vírgula ou aspa dentro de campo quebra `cut`, `awk` e `sed` ao mesmo tempo.** Use `mlr` ou `csvkit` — ou converta para JSON e volte ao `jq`.
- ✓ **NDJSON é o formato do ferramental moderno**, e o `jq` o processa nativamente; `--slurp` junta tudo num array quando a agregação exigir.

NO KALI

A saída JSON do ferramental moderno é feita para o `jq`. O `nmap` produz XML (`-oX`), mas `nuclei`, `httpx`, `subfinder` e `naabu` emitem **NDJSON — um achado por linha**. `nuclei -jsonl -o achados.jsonl` seguido de `jq -r 'select(.info.severity=="critical") | "\(.host) \(.info.name)"' achados.jsonl` transforma centenas de linhas no resumo do que precisa de atenção imediata. Isso é **triagem de resultado de teste autorizado** — a mesma ferramenta contra alvo sem contrato não muda o enquadramento da **Lei 12.737/2012**. **Converter o XML do `nmap` uma vez economiza o resto do trabalho:** `xmlstarlet sel -t -m '//host' -v 'address/@addr' -o " " -v 'ports/port[state/@state="open"]/@portid' -n varredura.xml` produz host e portas em texto simples, pronto para `sort`, `uniq` e `awk`. **`jq -e` transforma a checagem em condição de script:** `if jq -e '[.[] | select(.severity=="critical")] | length > 0' achados.json >/dev/null; then notificar; fi` só dispara o alerta quando há achado crítico — é a forma correta de encadear a triagem com o resto da automação, **sem `grep` sobre JSON. E confira a versão antes de assumir sintaxe:** o `jq` 1.6 e o 1.7 diferem em detalhes que mordem — o 1.7 corrigiu o arredondamento de números grandes, mudou a ordenação de `keys` para chaves não ASCII e acrescentou `pick`, `abs` e `toarray`. Um filtro escrito para o 1.7 falha num host mais antigo; `jq --version` resolve a dúvida em um segundo.